

The background is a collage. The top left features a globe with a blue and white grid pattern. The bottom right shows a close-up of a calculator's keypad with buttons labeled with numbers, letters, and symbols like *, /, %, and =. A hand is visible pressing one of the buttons. The entire image has a soft, blurred effect.

Machine Architecture

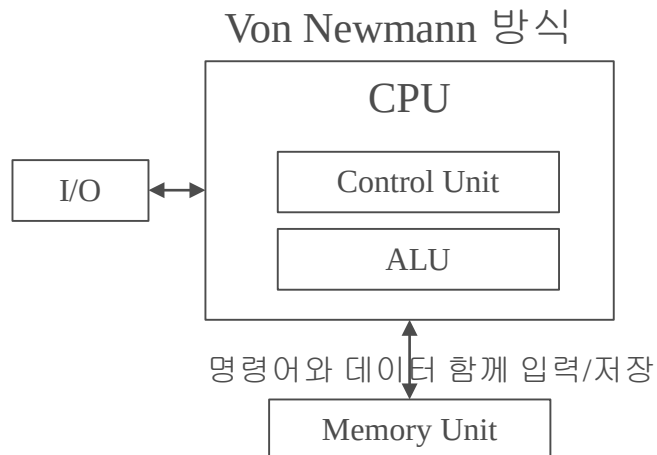
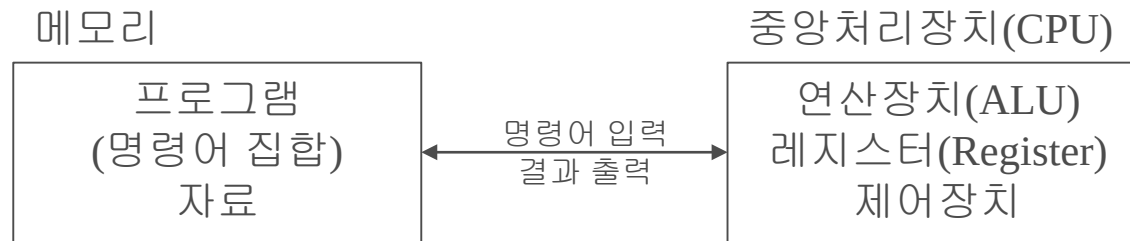
수업 목표

- 시스템 소프트웨어의 이해를 위한 기본 사항을 이해한다.
- 시스템 소프트웨어를 위한 컴퓨터 시스템 구성의 기본 개념을 이해한다.
- 시스템 소프트웨어와 컴퓨터 시스템과의 관계를 이해한다.

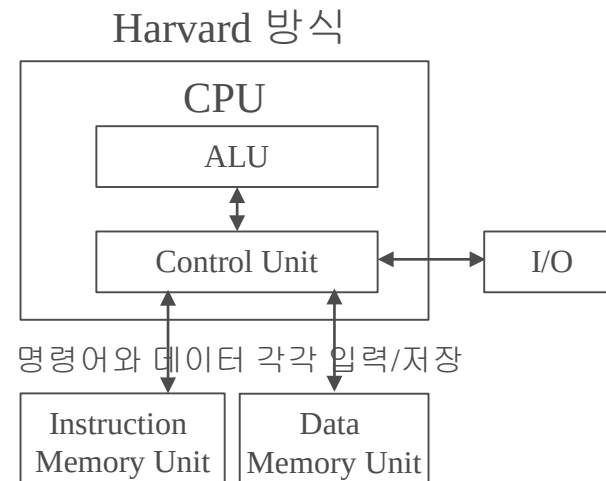
프로그램의 실행

- 저장 프로그램(Stored Program) 방식

- 폰 노이만(Von Neumann)이 고안
- 컴퓨터시스템을 구성하는 구성요소간의 유기적 연결로 프로그램 실행
- 메모리에 자료와 프로그램이 함께 저장



VS.



명령어 처리 과정

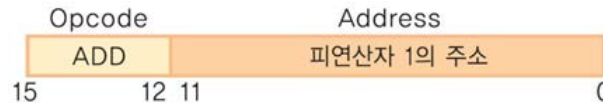
CPU와 주기억장치 간의 다양한 자료 전송이 발생

• 명령어 처리과정

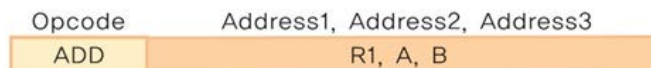
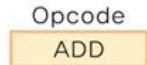
- 메모리의 명령어(instructions)와 자료(data)를 CPU의 여러 임시 저장 장소인 레지스터(registers)로 전송
- 명령어를 처리한 후
- 다시 처리 결과인 자료가 주기억장치로 전송되는 과정을 거침
- 메모리로부터 Fetch -> Decode -> Execute -> Store의 Cycle로 명령어 처리

프로그램 시행을 위한 명령어 형식

- 명령어(instruction)는 연산 부분(operation part)과 피연산 부분(operand part)으로 구성
 - 연산 부분은 명령어가 수행해야 할 기능을 의미하는 코드
 - 피연산 부분은 연산에 참여하는 자료를 의미하는 코드



- 명령어가 16비트로 구성
 - 4비트는 연산 종류(opcode)
 - 12비트는 피연산자의 메모리 주소(address)
 - 피연산자의 수에 따라 다양한 형태의 명령어가 존재,
 - 주소가 없거나 2개 또는 3개 메모리 주소 또는 레지스터



명령어 종류

- **Operator**

- 연산 또는 기호 단어(mnemonic symbol)를 이용
 - ADD (add)
 - LDA (load address)
 - STA (store address)
 - HLT (halt)

- **Operand**

- A, B, C 등으로 피연산자를 기술
- 해당 피연산자의 위치가 메모리의 주소를 나타냄

프로그래밍 언어

- 기계어

- 컴퓨터 이해할 수 있는 0과 1로 나타낸 컴퓨터 고유 명령 형식 언어

- 어셈블리 언어

- 컴퓨터 명령어인 기계어를 프로그래머가 좀 더 이해하고 사용하기 편하게 만든 프로그래밍 언어
- 명령어는 연산자와 피연산자로 구성

- 고급 언어

- High level language
(C, C++, Java, C# etc.)

명령어	어셈블리어	기계어
ADD	ADD A	0111000000000100
LDA	LDA B	0101000000000110
STA	STA C	0100000000000111
HLT	HLT	0011000000000000

기억장치

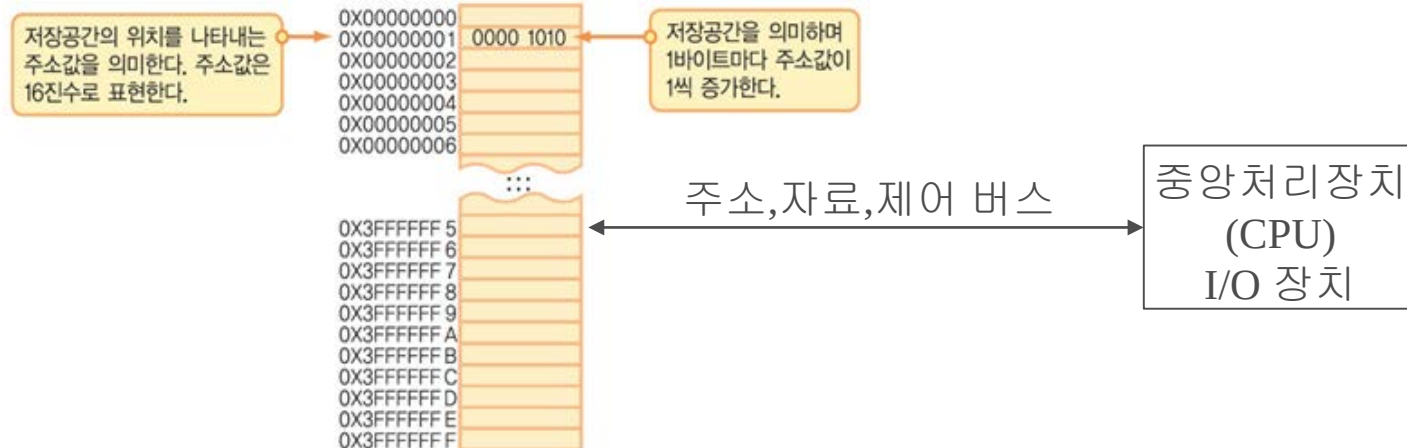
• 주기억장치의 구조

- 주소(Address)

- 메모리의 저장소는 주소(address)를 이용하여 각각 바이트(byte) 단위로 고유하게 식별
- 컴퓨터가 한 번에 작업할 수 있는 데이터의 단위를 워드(word)
- 워드는 32비트 또는 64비트

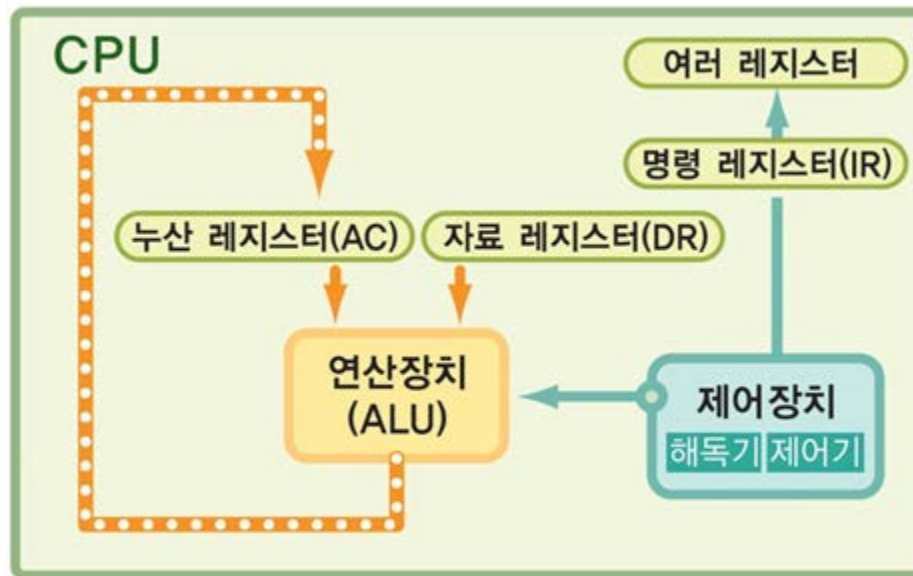
- 버스(Bus)

- 관련 자료 전달 경로
- Address bus(물리주소 지정하는데 필요한 신호 전달) - MAR(Memory Address Register)과 연결
- Data bus (외부버스, 데이터 전달) - MBR(Memory Buffer Register)과 연결
- Control bus (R/W 명령제어 신호 전달)



중앙처리장치 (CPU)

- 메모리에서 필요한 자료를 이용하여, 저장된 명령어를 순차적 (Sequential)으로 실행
- 연산장치: 자료의 연산을 수행
- 제어장치: 컴퓨터의 작동을 제어
- 레지스터: 연산에 필요한 자료를 임시로 저장



레지스터(Register)

- CPU는 컴퓨터가 명령의 수행 과정을 처리하기 위한 임시 기억 장치
 - CPU 내의 레지스터 크기와 수는 중앙처리장치의 성능에 매우 중요한 요소이므로 가격과 성능을 고려하여 결정

레지스터 심볼	레지스터 이름	기능
DR	자료 레지스터 (Data Register)	연산에 필요한 피연산자를 저장하는 레지스터
AR	주소 레지스터 (Address Register)	현재 접근할 기억장소의 주소를 기억하는 레지스터
AC	누산 레지스터 (Accumulator Register)	연산장치의 입출력 데이터를 임시적으로 기억하는 레지스터
IR	명령어 레지스터 (Instruction Register)	현재 수행 중인 명령어를 저장하고 있는 레지스터
PC	프로그램 카운터 (Program Counter)	다음에 실행할 명령어의 메모리 주소가 저장된 레지스터
TR	임시 레지스터 (Temporary Register)	임시로 자료를 저장하는 레지스터로 범용 레지스터라고도 부름

CPU의 연산장치와 제어장치

- 연산장치

- 중앙처리장치의 임시기억장소인 누산 레지스터(AC - Accumulator)와 자료 레지스터(DR - Data Register)에 저장된 자료를 연산에 참여할 때 연산자로 이용
- 결과는 다시 누산 레지스터에 저장되어 필요하면 주기억장치에 저장되거나 다른 연산에 이용
 - $AC \leftarrow AC + DR$
- 두 레지스터 피연산자의 연산을 연산장치(ALU)가 제어장치(CU)의 신호를 받아 실행

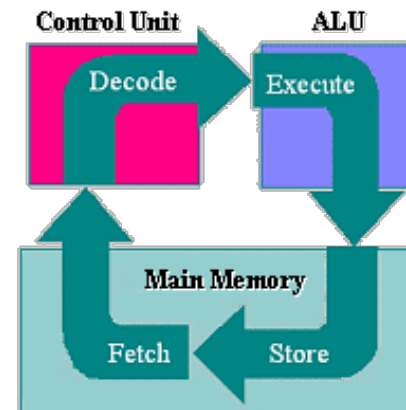
- 제어장치(Control Unit)

- 연산장치(ALU)에서의 산술 및 논리 연산에 요구되는 작업을 연속적으로 수행하는 신호를 보냄으로써 ALU와 레지스터가 명령(instruction)을 수행하게 하는 장치
- 여러 개의 해독기(decoder)와 제어기(controller)로 구성

기계 주기(machine cycle)

• 기계 주기(Machine Cycle)

- CPU는 하나의 명령어를 실행하기 위해 인출, 해독, 실행의 세 과정을 거침
- 인출(Fetch)
 - 제어 장치가 프로그램 카운터(PC)에 있는 메모리 주소를 참조해 메모리로부터 다음 수행할 명령어를 가져와 명령 레지스터(IR)에 저장
 - 다음 명령어를 수행하기 위해서 PC를 하나 증가 시킴
- 해독(Decode)
 - 제어 장치는 명령 레지스터(IR)에 있는 명령어를 연산 부분(operation part)과 피연산 부분(operand part)으로 해독
 - 만일 명령어가 피연산 부분이 있는 명령어라면 피연산 부분의 메모리 주소를 주소 레지스터(AR)에 저장
- 실행(Execution)
 - 중앙처리장치는 각 구성 요소에게 작업 지시
 - 다음 명령어로 다시 기계 주기를 반복



프로그램 작성: 두 정수 합 구하기

- 만일 두 수가 각각 32와 -18이라면
 - 이때, 기호 A는 32를 의미하며, 기호 B는 -18을 의미
 - 메모리에 더 작은 단위의 여러 명령어 집합으로 구성하여 그 명령을 실행
 - 두 정수의 합을 구하기 위해서는 다음과 같이 4개의 명령어 집합으로 가능

순서	명령어	의미
명령어1	LDA A	메모리 A의 내용을 누산 레지스터(AC)에 저장
명령어2	ADD B	메모리 B의 내용과 누산 레지스터(AC)의 값을 더하여 다시 누산 레지스터(AC)에 저장
명령어3	STA C	누산 레지스터(AC)의 값을 메모리 C에 저장
명령어4	HLT	프로그램 종료

두 정수 합 구하기: 명령어의 세부 수행

- 명령어 LDA의 기능

- 주소 레지스터(**AR**)의 주소 값을 갖는 메모리 자료(**M[AR]**)를 누산 레지스터(**AC**)에 저장
- 이 처리를 위하여 자료 레지스터(**DR**)를 다시 누산 레지스터에 저장

- 명령어 ADD, STA

- 피연산자(Operand)

- Operand A
 - 메모리 주소 0012FF40에 저장된 32
- Operand B
 - 명령어 ADD B에서 메모리 주소 0012FF44에 저장된 -18
- Operand C
 - 명령어 STA C에서 피연산자 C는 메모리 주소 0012FF48을 가리키며
 - 여기에는 $32 + (-18)$ 의 결과인 14가 저장



이 동영상 교과목 자료는
GJ-RIP 사업의
지자체-대학 협력기반 지역혁신 사업비로 개발되었음.